Descripción y justificación del problema

Taller 01: Simulación de MANETs

ME01 — Modelos estocásticos y simulación en computación y comunicaciones

Universidad Nacional de Colombia

1. Contexto: el perro perdido y su collar

El escenario de estudio es sencillo de enunciar. Un perro se extravía llevando un collar que emite una señal (beacon) una vez por segundo. Los celulares que pasan cerca conocen su propia ubicación por GPS y miden con qué intensidad reciben la señal del collar (RSSI, Received Signal Strength Indicator). A partir de esas lecturas, la posición del observador y la potencia recibida, la red estima dónde está el perro, y el dueño consulta esa estimación a través de la misma red. La idea es análoga a servicios comerciales de localización colaborativa como Find My de Apple, y aquí sirve de excusa concreta para estudiar modelos de movilidad en una red ad hoc móvil (MANET) [1].

Lo interesante del problema no es tanto el collar como la forma en que se mueven los observadores. Los celulares no deambulan de manera independiente: viajan en grupos que se desplazan como un bloque, los pasajeros de un bus, la gente reunida en una plaza, un paseador con su grupo de perros, y dentro de cada grupo cada persona se mueve un poco por su cuenta. Esa estructura motiva un sistema jerárquico de tres capas, en el que la posición absoluta de cada celular es la suma de una zona fija (sur, con y entre 0 y 150 m, y norte, con y entre 150 y 300 m), un clúster que se desplaza como bloque (cuatro en total: bus-norte y plaza en el norte, bus-sur y paseador en el sur) y el propio celular, que camina al azar dentro de la caja de su clúster (8 por clúster, 32 en total).

El área simulada es de 500 × 300 m. Los buses recorren una ruta rectangular a 6 m/s con paradas de 20 s en cada esquina, y los grupos a pie van de un punto al azar a otro. El perro es un nodo aparte, que no pertenece a ningún clúster y camina al azar a 0.3 m/s. Toda la simulación se construye en NS-3 3.46.1 (C++) [2], [4], y el análisis posterior en Python.

Esta jerarquía cumple la regla del taller de tener al menos tres clústeres en el primer nivel y dos en el segundo, con movimiento tanto de clústeres como de nodos. Y plantea la tensión central del problema: moverse ayuda a oír, pero puede dañar la geometría de la medición.

2. Pregunta de investigación

> ¿Cómo afecta el patrón de movilidad de los celulares que observan al error y a la frescura de la posición estimada de un collar por RSSI?

Es una pregunta sobre modelos de movilidad en una MANET, de dificultad intermedia, que se responde con el método científico: se declara una hipótesis, se diseña un experimento controlado que varía el patrón de movilidad, se mide el efecto sobre variables de salida bien definidas y se concluye a partir de esas mediciones con sus réplicas.

3. Hipótesis

El patrón de movilidad empuja en dos direcciones opuestas, y de esa tensión surge la hipótesis. A favor de la frescura, cuando los clústeres se mueven los observadores cubren más área y pasan cerca del collar con más frecuencia, de modo que llegan lecturas más a menudo y la posición estimada queda más reciente, con menor edad. En contra de la precisión, los celulares de un mismo bus están casi en el mismo punto del espacio, y para trilaterar eso es una geometría pobre: muchas lecturas tomadas desde casi la misma posición aportan poca información para fijar un punto, por lo que el error de localización puede crecer aunque haya abundancia de datos.

La hipótesis, entonces, es que la movilidad mejora la frescura pero puede degradar la precisión geométrica, y que el resultado neto depende de si el movimiento diversifica las posiciones de observación o solo traslada en bloque un conjunto de observadores apiñados. El experimento del literal f debe permitir discriminar cuál efecto domina en cada escenario de movilidad.

4. Por qué importa y por qué la movilidad es la variable de estudio

La localización colaborativa por RSSI es barata: no requiere infraestructura dedicada ni hardware especial más allá de los celulares que ya circulan. Su calidad, sin embargo, depende de quién oye el collar y desde dónde, y eso lo determina la movilidad de la multitud de observadores. Entender cómo la movilidad condiciona el error y la frescura es el núcleo del problema: es la variable que el diseñador del sistema no controla, porque la gente se mueve como quiere, pero cuyo efecto debe anticipar.

Se elige la movilidad como variable independiente principal por tres razones. Primero, es la que fija el enunciado del taller. Segundo, es la que produce la tensión no trivial de la Sección 3: a diferencia de subir la densidad de celulares, que casi siempre ayuda, moverse puede ayudar y perjudicar a la vez, lo que vuelve la pregunta genuinamente abierta. Tercero, es la que mejor se presta a un diseño factorial limpio, porque el código expone la movilidad como dos interruptores independientes, uno de clúster y otro de nodo, cuyas cuatro combinaciones son escenarios cualitativamente distintos.

Estudiar el comportamiento de una red ad hoc jerárquica variando de forma controlada un parámetro de diseño, y cuantificar su efecto con un modelo, es el tipo de indagación que ilustra el trabajo de Ospina-López y Ortiz-Triviño sobre un factor de crecimiento para redes ad hoc escalables [3]: allí también se parte de un sistema jerárquico y se busca una relación medible entre un parámetro de la red y su desempeño. Nuestra pregunta es más modesta en alcance, pero comparte el método.

5. Variables

El código de la simulación expone todos los parámetros en `codigo/escenarios/base.conf`. A partir de ellos se organizan las variables del estudio en tres grupos: las que se cambian (independientes), las que se miden (dependientes) y las que se dejan fijas.

5.1 Variables independientes

La variable de interés es el patrón de movilidad, expresado como dos factores binarios:

Factor	**Parámetro en `base.conf`**	**Valores**	**Qué controla**
Movilidad de clúster	`clusterMobility`	`true` / `false`	si los clústeres (nivel 1) se desplazan como bloque

Factor	Parámetro en `base.conf`	Valores	Qué controla
Movilidad de nodo	`nodeMobility`	`true` / `false`	si cada celular camina dentro de la caja de su clúster

Sus cuatro combinaciones definen los escenarios de movilidad del experimento: estático (ambos `false`), solo-nodo, solo-clúster y clúster+nodo (ambos `true`, el valor por defecto).

Hay además un conjunto de factores secundarios, también variables desde `base.conf`, que permiten estudiar si el efecto de la movilidad es robusto o depende del contexto:

Factor secundario	Parámetro	Valores	Papel en el estudio
Protocolo de enrutamiento	`routing`	`aodv` [6] / `olsr` [7] / `dsdv` [8]	cómo viajan los mensajes por la red
Fusión de zonas	`zoneFusion`	`none` / `raw` / `merge`	cómo se combinan las dos zonas al estimar
Método de estimación	`estMethod`	`cercano` / `centroide` / `trilateracion`	cómo se convierten las lecturas en una posición
Densidad	`nodesPerCluster`	entero (por defecto 8)	cuántos observadores hay por clúster
Sombreado	`shadowSigmaDb`	dB (por defecto 4; 0 = sin sombreado)	magnitud del ruido aleatorio del canal

El diseño del literal f puede tratar algunos de estos factores como parte del factorial, por ejemplo cruzar movilidad con protocolo, y fijar los demás según el presupuesto de corridas.

5.2 Variables dependientes (lo que se mide)

Variable medida	Unidad	De dónde sale
Error de localización	m	distancia entre la posición estimada y la real (`estimaciones_red.csv`, `consultas.csv`)
Frescura o edad de la estimación	s	antigüedad de la estimación cuando se responde una consulta (`consultas.csv`)
Consultas respondidas	tasa (fracción)	si el dueño recibió respuesta y con qué demora (`consultas.csv`)

El error y la frescura son las dos salidas que nombra la pregunta. La tasa de consultas respondidas completa el panorama, porque una estimación excelente no sirve de nada si no llega al dueño, y esa entrega depende a su vez del protocolo y de la movilidad.

5.3 Variables fijas (controladas)

Para aislar el efecto de la movilidad, el resto de parámetros se mantienen en sus valores por defecto de `base.conf`, salvo que un factor los declare de forma explícita. Entre ellos están la duración (`simTime=600` s), el collar y el canal (`beaconInterval=1` s, `collarTxDbm=0` dBm, `pathLossExp=2.7`, `refLossDb=40.05` dB, `gpsSigma=4` m), la posición y velocidad del perro (`dogX=300`, `dogY=140`, `dogSpeed=0.3` m/s) y los parámetros de estimación y consulta (`estWindow=10` s, `estInterval=5` s, `queryInterval=10` s, `ownerCluster=0`). Mantenerlos fijos es lo que permite atribuir las diferencias observadas al factor que se está variando.

6. Método: cómo se responde la pregunta

La pregunta se responde con un diseño factorial con réplicas, siguiendo el enfoque de diseño de experimentos y análisis de resultados para simulaciones de redes ad hoc del material del curso [2]. El procedimiento es el siguiente.

1. Factores y niveles. El factor principal es la movilidad, con cuatro escenarios, cruzado con uno o más factores secundarios, típicamente el protocolo de enrutamiento. Cada combinación de niveles es un tratamiento.
2. Réplicas con semillas distintas. Cada tratamiento se corre varias veces cambiando la semilla del generador aleatorio (`RngRun`), porque el sombreado del canal y la movilidad al azar introducen variabilidad. Las réplicas permiten estimar la media de cada salida con su incertidumbre (intervalos de confianza) en lugar de quedarse con una sola corrida.
3. Mediciones. De cada corrida se extraen el error, la frescura y la tasa de consultas respondidas (Sección 5.2), a partir de los archivos CSV que escribe la simulación.
4. Comparación. Se comparan las medias entre escenarios de movilidad para contrastar la hipótesis de la Sección 3: ver si moverse mejora la frescura, si daña o no la precisión y bajo qué condiciones domina cada efecto.

La ejecución del factorial está automatizada en `analisis/experimento.py`, que corre las cuatro combinaciones de movilidad con cada protocolo, repite cada una con semillas distintas y deja las tablas de resultados en `resultados/<nombre>/`. Cada corrida guarda sus parámetros y su semilla, de modo que cualquier resultado es reproducible.

El diseño detallado del experimento, las corridas efectivas y los resultados numéricos corresponden al literal f. Este documento solo plantea el problema y el método; no adelanta cifras ni conclusiones.

7. Alcance y supuestos

La simulación es un modelo y, como todo modelo, hace simplificaciones. Para interpretar bien los resultados conviene declararlas por adelantado; son supuestos y artefactos del modelo, no conclusiones [2].

El radio es 802.11b en lugar de Bluetooth. Un collar real usaría Bluetooth de baja energía, pero ns-3 no lo modela, así que se usa Wi-Fi ad hoc 802.11b como sustituto [4]. Esto cambia las magnitudes absolutas de alcance y potencia, no la lógica del estudio.

El sombreado se sortea independiente por paquete. En cada beacon el sombreado del canal se sortea de forma independiente, cuando en la realidad está correlacionado en el espacio y en el tiempo [1]. En consecuencia, promediar lecturas ayuda más en la simulación que en la vida real.

Las lecturas lejanas están sesgadas. Lejos del collar solo se decodifican los beacons que tuvieron un sombreado favorable, porque los débiles se pierden bajo el umbral de sensibilidad. Las lecturas lejanas salen así más fuertes de lo que deberían: al ajustar RSSI contra distancia, el exponente aparente baja a cerca de 1.6, frente al 2.7 del modelo, y la trilateración subestima distancias. Sin sombreado (`shadowSigmaDb=0`) el ajuste recupera el 2.7 exacto.

La pila de red introduce artefactos. ns-3 guarda solo 3 paquetes por vecino mientras resuelve direcciones con ARP; los relevos salen en ráfaga, por lo que con AODV se perdía cerca del 40 %. Subir esa cola a 64 (`arpQueue`) elevó la entrega con AODV del 57 % al 93 %. Además, las rutas de AODV vencen a los 3 s mientras las consultas llegan cada 10 s aproximadamente, lo que dejaba cerca del 22 % de las consultas sin respuesta; con OLSR y DSDV, entre 1 % y 7 %. Son observaciones de las pruebas de desarrollo, útiles como contexto para elegir y comparar protocolos.

La fusión `none` sesga la estimación. Con el valor por defecto, cada cabeza de zona solo ve las lecturas de su lado y su estimación se carga hacia su zona. Las alternativas `raw` y `merge` corrigen eso a costa de más tráfico; `raw` llegó a bajar la entrega de relevos del 99 % al 67 % en una prueba, por lo que el diseño debe tener en cuenta ese compromiso.

Hay cosas que quedan por fuera. No se mide la distancia por tiempo de vuelo (Wi-Fi FTM), que ns-3 no modela bien, y queda como posible trabajo futuro. Tampoco se modelan paradas donde suban o bajen pasajeros: los mismos 8 celulares acompañan a cada clúster durante toda la corrida.

Estos límites acotan el alcance del estudio: las conclusiones valen dentro del modelo descrito y deben leerse teniendo presentes los artefactos anteriores.

8. Referencias

[1] S. K. Sarkar, T. G. Basavaraju, y C. Puttamadappa, *Ad Hoc Mobile Wireless Networks: Principles, Protocols, and Applications*, 2.ª ed. Boca Raton, FL, EE. UU.: CRC Press, 2013.

[2] H. Zárate Ceballos, J. Parra Amaris, H. Jiménez Jiménez, D. Romero Rincón, O. Agudelo Rojas, y J. E. Ortiz Triviño, *Wireless Network Simulation: A Guide using Ad Hoc Networks and the ns-3 Simulator*.

[3] J. P. Ospina-López y J. E. Ortiz-Triviño, "Estimation of a growth factor to achieve scalable ad hoc networks", *Ingeniería y Universidad*, vol. 21, n.º 1, pp. 49–70, 2017, doi: 10.11144/Javeriana.iyu21-1.egfa.

[4] nsnam, "ns-3 Network Simulator", versión 3.46. [En línea]. Disponible: https://www.nsnam.org/

[6] C. Perkins, E. Belding-Royer, y S. Das, "Ad hoc On-Demand Distance Vector (AODV) Routing", IETF, RFC 3561, jul. 2003.

[7] T. Clausen y P. Jacquet, "Optimized Link State Routing Protocol (OLSR)", IETF, RFC 3626, oct. 2003.

[8] C. E. Perkins y P. Bhagwat, "Highly dynamic Destination-Sequenced Distance-Vector routing (DSDV) for mobile computers", en *Proc. ACM SIGCOMM*, 1994, pp. 234–244.